Polymorphism in C++

Polymorphism in C++ is an object-oriented programming concept that allows entities such as functions or objects to take on multiple forms. It enables the same interface to be used for different underlying data types, making code more flexible, reusable, and easier to maintain[137].

Types of Polymorphism in C++

1. Compile-Time Polymorphism (Static Polymorphism):

- **Function Overloading:** Multiple functions with the same name but different parameter lists exist in the same scope. The compiler determines which function to call based on the arguments at compile time

- **Operator Overloading:** Operators (like +, *, etc.) are redefined to work with user-defined types. For example, the + operator can add integers or concatenate strings

- **Templates:** Allow classes and functions to operate with generic types without rewriting code for each type

Example: Function Overloading

```
class Shape {
public:
    void draw() { std::cout << "Drawing a shape\n"; }
    void draw(int sides) { std::cout << "Drawing a shape with " << sides << "
sides\n"; }
};
// The compiler chooses the correct draw() based on arguments at compile time[7].
```

2. Run-Time Polymorphism (Dynamic Polymorphism):

- Achieved through **inheritance** and **virtual functions**.

- A base class declares a virtual function, and derived classes override it.
- The function to execute is determined at runtime based on the actual object type, even if accessed through a base class pointer or reference

Example: Virtual Functions

```
class Animal {
public:
    virtual void animalSound() { std::cout << "The animal makes a sound\n"; }
};

class Dog : public Animal {
public:
    void animalSound() override { std::cout << "The dog says: bow wow\n"; }
};

Animal* a = new Dog();
a->animalSound(); // Output: The dog says: bow wow (resolved at runtime)
```

Benefits of Polymorphism

- **Code Reusability:** Write generic code that works with different types
- **Flexibility:** Easily extend programs by adding new classes with minimal changes to existing code.
- **Interface-Based Programming:** Enables programming to interfaces rather than specific implementation.
- **Abstraction:** Hides implementation details and exposes only necessary interfaces.

key differences between method overloading and method overriding in object-oriented programming :

Feature	Method Overloading	Method Overriding
Definition	Multiple methods with the same name but different parameter lists in a class	A subclass provides a specific implementation of a method already defined in its superclass
Where it occurs	Within a single class	Between a parent (superclass) and a child (subclass) class
Method Signature	Methods must differ in number, type, or order of parameters	Methods must have exactly the same signature (name, parameters, order, and type)
Return Type	Can be the same or different	Must be the same or a subtype (covariant return type)
Access Modifier	No restriction; can use any access modifier	Overriding method must have the same or less restrictive access modifier
Inheritance Required	Not required	Required (must involve inheritance)
Binding Time	Compile-time (static binding)	Runtime (dynamic binding)
Polymorphism Type	Compile-time polymorphism	Runtime polymorphism
Purpose	To provide multiple ways to perform similar operations with different inputs	To modify or extend the behavior of an inherited method

Private/Static/Final Methods	Can be overloaded	Cannot be overridden
Error Handling	Errors detected at compile time	Errors may occur at runtime
Usage	Improves code readability and usability by allowing the same method name for similar actions	Enables subclass-specific behavior and supports polymorphism

---

Limitations of method overloading:

- **Cannot Overload by Return Type Alone:**
  You cannot create overloaded methods that differ only in their return type. The parameter list must be different; otherwise, the compiler will throw an error

- **Methods Must Have Different Signatures:**
  Overloaded methods must differ in the number, type, or order of their parameters. If two methods have the same name and parameter list, even with different return types, it leads to a compile-time error

- **All Overloaded Methods Must Be in the Same Class:**
  Method overloading is only possible within the same class. Methods with the same name in different classes are not considered overloaded

- **Potential for Ambiguous Calls:**
  If overloaded methods have similar parameter types or use automatic type promotion, it can lead to ambiguity, making it unclear which method will be called. This can result in compile-time errors or unintended behavior

- **Complexity and Maintainability Issues:**
  Excessive overloading can make code harder to read and maintain. When many overloaded methods exist, it may become confusing for developers to know which method to use, increasing the risk of mistakes and misunderstandings

- **No Overloading of Static Methods with Same Signature:**
  Static methods cannot be overloaded if they have the same name and parameter types, even if declared in different scopes

- **Does Not Support Overloading by Access Modifier:**
  Changing only the access modifier (e.g., public, private) does not constitute overloading; the parameter list must still be different.

- **Possible Concealed Semantics:**
  Having multiple methods with the same name can hide the actual intent or semantics of the code, making it harder to understand and debug

Polymorphism Classification

A. Compile-Time Polymorphism

- **Includes**: Ad hoc (overloading) and parametric (templates/generics).
- **Binding**: Static.
- **Performance**: Faster (no runtime overhead).

B. Runtime Polymorphism

- **Includes**: Subtyping (virtual functions/method overriding).
- **Binding**: Dynamic.
- **Flexibility**: Enables late binding for extensible code.
-

Operator Overloading in C++

Operator overloading in C++ allows you to redefine the way operators work for user-defined types (classes and structs). This makes it possible to use operators such as `+`, `-`, `*`, `++`, etc., with objects, enabling intuitive and readable code for custom data types

Why Use Operator Overloading?

- Provides natural syntax for operations on user-defined types (e.g., adding two complex numbers with + instead of a function call)
- Makes code more readable and maintainable.
- Allows custom behaviors for operators when used with objects.

Syntax and Examples

Unary Operator Overloading

Overloading the prefix ++ operator for a class:

```
class Count {
private:
    int value;
public:
    Count() : value(5) {}
    // Overload prefix ++ operator
    void operator++() {
        ++value;
    }
    void display() {
        std::cout << "Count: " << value << std::endl;
    }
};
```

```
int main() {
    Count count1;
    ++count1;
    count1.display(); // Output: Count: 6
    return 0;
}
```

Binary Operator Overloading

Overloading the + operator for a Complex class:

```
class Complex {
private:
    float real, img;
public:
    Complex() : real(0), img(0) {}
    Complex operator+(const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.img = img + obj.img;
        return temp;
    }
    void display() {
        std::cout << real << " + " << img << "i" << std::endl;
    }
};
```

```
int main() {
    Complex c1, c2, result;
    // Assume c1 and c2 are initialized
    result = c1 + c2;
    result.display();
    return 0;
}
```

Operators That Can and Cannot Be Overloaded

Can be overloaded:

- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Assignment: `=`, `+=`, `-=`, etc.
- Comparison: `==`, `!=`, `<`, `>`, etc.
- Increment/Decrement: `++`, `--`
- Subscript: `[]`
- Function call: `()`
- Member access: `->`
- Bitwise: `&`, `|`, `^`, `~`, `<<`, `>>`
- Memory: `new`, `delete`

Cannot be overloaded:

- Scope resolution: `::`
- Member access: `.`
- Member pointer selector: `.*`
- Ternary conditional: `?:`
- `sizeof` operator

Advantages

- Enables natural and readable syntax for custom types.
- Makes user-defined types behave like built-in types.
- Improves code clarity, especially in mathematical or domain-specific applications

Limitations

- Cannot change operator precedence or arity.
- Only existing operators can be overloaded, not new ones.
- Overuse can reduce code clarity if not used judiciously

Constructor Overloading in C++

Constructor overloading in C++ allows a class to have multiple constructors with the same name (the class name) but different parameter lists. This enables objects of a class to be initialized in different ways, depending on the arguments provided at the time of object creation.

Key Points

- **Same Name, Different Parameters:**
  All constructors must have the same name as the class, but each must differ in the number or type of parameters

- **Flexibility:**
  Allows objects to be created with different initial values or states, increasing the flexibility and readability of code.

- **Automatic Selection:**
  The compiler automatically calls the constructor whose parameter list matches the arguments provided during object instantiation.

Example

```
#include <iostream>
using namespace std;

class Box {
public:
    double length, breadth, height;

    // Default constructor
    Box() : length(1), breadth(1), height(1) {
        cout << "Default constructor called" << endl;
    }
    // Constructor with one parameter (cube)
    Box(double side) : length(side), breadth(side), height(side) {
        cout << "Constructor with one parameter called" << endl;
    }
    // Constructor with three parameters
    Box(double l, double b, double h) : length(l), breadth(b), height(h) {
```

```
        cout << "Constructor with three parameters called" << endl;
    }

    double volume() {
        return length * breadth * height;
    }
};

int main() {
    Box box1;               // Calls default constructor
    Box box2(5);            // Calls constructor with one parameter
    Box box3(2, 3, 4);      // Calls constructor with three parameters

    cout << "Volume of box1: " << box1.volume() << endl;
    cout << "Volume of box2: " << box2.volume() << endl;
    cout << "Volume of box3: " << box3.volume() << endl;

    return 0;
}
```

Output:

text

```
Default constructor called
Constructor with one parameter called
Constructor with three parameters called
Volume of box1: 1
Volume of box2: 125
Volume of box3: 24
```

Rules for Constructor Overloading

- Each constructor must have a unique parameter list (different number or types of parameters).
- Constructors are always defined within the same class.
- Overloading cannot be done by changing only the return type.
- Both default and parameterized constructors can coexist in the same class.

Advantages

- **Convenience:** Multiple ways to initialize objects.
- **Readability:** The intent of object creation is clear from the constructor used.
- **Flexibility:** Supports different initialization scenarios without extra setter methods.

Object Pointers in C++

Object pointers in C++ store the memory address of objects (instances of classes). They enable dynamic memory management, efficient object manipulation, and polymorphism. Here's a structured breakdown:

1. Declaration & Initialization

- **Syntax**:

```
ClassName *ptr;      // Declare a pointer to an object of ClassName
ptr = &existingObj;  // Assign address of an existing object
```

-
- Or dynamically:

```
ClassName *ptr = new ClassName();  // Allocate memory and call constructor
```

-

2. Accessing Members

- **Arrow Operator (`->`)**:
  Used to access members (variables/functions) of an object via its pointer:

  cpp

```
ptr->memberFunction(); // Call member function
ptr->dataMember = 10;  // Access data member
```

-
- **Dereferencing (`*`)**:
  Access the object itself, then use the dot operator (`.`):

  cpp

```
(*ptr).memberFunction(); // Equivalent to ptr->memberFunction()
```

-

3. Dynamic Memory Allocation

- `new` **Operator**:
  Allocates memory on the heap and returns a pointer to the object:

  cpp

```
MyClass *objPtr = new MyClass(); // Calls constructor
```

-
- `delete` **Operator**:
  Frees memory to prevent leaks:

  cpp

```
delete objPtr; // Calls destructor
objPtr = nullptr; // Prevents dangling pointer
```

-

4. Example Code

```
#include <iostream>
using namespace std;

class Complex {
public:
    float real, imag;
    void setData(float r, float i) { real = r; imag = i; }
    void display() { cout << real << " + " << imag << "i\n"; }
};

int main() {
    // Static object
    Complex obj1;
    Complex *ptr1 = &obj1;
    ptr1->setData(3.5, 2.1);
    ptr1->display(); // Output: 3.5 + 2.1i

    // Dynamic object
    Complex *ptr2 = new Complex();
    ptr2->setData(1.0, 4.5);
    ptr2->display(); // Output: 1.0 + 4.5i
    delete ptr2;
```

```
    return 0;
}
```

5. Key Use Cases

- **Polymorphism**:
  Base class pointers can point to derived class objects for runtime polymorphism:

```
class Base { public: virtual void show() { /*...*/ } };
class Derived : public Base { public: void show() override { /*...*/ } };

Base *bPtr = new Derived();
bPtr->show(); // Calls Derived::show(
```

- **Arrays of Objects**:

```
MyClass *arr = new MyClass[5]; // Array of 5 objects
delete[] arr; // Free array memory
```

- **Passing to Functions**:
  Efficiently pass large objects without copying:

```
void processObject(MyClass *obj) { /*...*/ }
```

-

Common Pitfalls

- **Dangling Pointers**:
  Occurs when a pointer points to freed memory. Always set pointers to `nullptr` after deletion.

- **Memory Leaks**:
  Forgetting to `delete` dynamically allocated objects.

- **Null Pointer Dereference**:
  Always check `if (ptr != nullptr)` before accessing members.

`this` Pointer in C++

The `this` pointer is an **implicit, hidden pointer** available in all non-static member functions of a class. It holds the **memory address of the current object** and is used to access its members, resolve naming conflicts, and enable method chaining.

Key Features

1. **Implicit Access**: Automatically passed to non-static member functions as a hidden argument

2. **Scope**: Available only within non-static member functions. **Static functions** do not have a `this` pointer

3. **Syntax**:

- Access members: `this->member` or `(*this).member`.
- Return the current object: `return *this;` (used for method chaining).

Common Use Cases

1. Resolving Name Conflicts

Differentiate between member variables and parameters with identical names:

cpp

```cpp
class Example {
    int x;
public:
    void setX(int x) {
        this->x = x; // 'this->x' refers to the member variable
    }
};
```

2. Method Chaining

Return `*this` to enable sequential function calls:

cpp

```cpp
class Coordinates {
    int lat, lon;
public:
    Coordinates& setLat(int lat) {
        this->lat = lat;
        return *this;
    }
```

```
    Coordinates& setLon(int lon) {
        this->lon = lon;
        return *this;
    }
};

int main() {
    Coordinates loc;
    loc.setLat(40).setLon(70); // Chained calls
}
```

3. Avoiding Self-Assignment

Check *this* address to prevent assigning an object to itself:

cpp

```
class Test {
    int data;
public:
    Test& operator=(const Test& rhs) {
        if (this != &rhs) { // Critical check
            data = rhs.data;
        }
        return *this;
    }
};
```

4. Accessing Members in Polymorphism

Ensure derived class overrides are called via base class pointers:

cpp

```cpp
class Base {
public:
    virtual void show() {
        cout << "Base: " << this << endl;
    }
};
class Derived : public Base {
public:
    void show() override {
        cout << "Derived: " << this << endl;
    }
};

int main() {
    Base* b = new Derived();
    b->show(); // Output depends on actual object type
}
```

Limitations

- **No *this* in Static Functions**: Static methods are class-level and lack object context.
- **Dangling References**: Returning **this* from a temporary object risks undefined behavior
- **Multiple Inheritance Ambiguity**: *this* may point to different base class subobjects, causing confusion

How the Compiler Handles *this*

When a member function is called, the compiler internally adds the *this* parameter:

cpp
```cpp
// Original code
obj.func();

// Compiler's interpretation
func(&obj);
```

The function *func* receives *this* as *&obj*

Example: Basic Usage

cpp
```cpp
#include <iostream>
using namespace std;

class Box {
    double length;
public:
    Box(double l) : length(l) {}
    void printAddress() {
        cout << "Object address: " << this << endl;
    }
};

int main() {
    Box b(5.0);
```

```
    b.printAddress(); // Output: Object address: 0x7ffd...
}
```